ChatGPT Queries

Idea and Datasets

Do you think that the the API design: "An environmental and urban climate statistics API that stores historical weather, air quality, and temperature data, enabling users to query aggregated metrics, anomalies, and city-level trends." is supported by sufficient data sources? Could you suggest some relevant datasets that are freely available online?

Short answer:

Yes — your idea is very well supported by available data, and it's a strong choice for a high-grade project.

Key point (most important)

Data **exists in abundance**, but it's:

- fragmented
- inconsistent
- from multiple sources

👉 The real challenge is **data integration and modelling**, not availability.

Good dataset options

Weather

- Met Office / historical UK weather datasets
- OpenWeatherMap (API for live + historical data)

Air Quality

- UK-AIR (DEFRA) – national pollutant data
- London Air – high-resolution urban data

Environmental

- UK government datasets (ONS, DEFRA)
- Environmental monitoring networks

Strengths of your idea

- Naturally supports CRUD + analytics
- Strong for trends, comparisons, anomalies
- High ceiling for marks if done well

Risks to manage

- Overcomplicating with too many datasets
- Data cleaning complexity
- Weak API design (just dumping data instead of meaningful endpoints)

What makes it high-grade

Focus on:

- Aggregations (averages, summaries)
- Trends over time
- Anomaly detection
- Comparisons between cities

Bottom line

- **Feasible?** Yes
- **Well-supported by data?** Definitely
- **High-grade potential?** Very high — if you handle data modelling and API design properly

---

If you want, next we can decide:
👉 single dataset (simpler) vs multi-source integration (harder, higher reward)

Architectural Design

For the implementation of an environmental statistics API, please compare the implementation of a city-centric design with other possible architectures. Please consider API resources, relationships, and techniques with REST, database schema, support for analytical queries, and suitability for a 10 minute demonstration. Please recommend one and explain the pros and cons.

A **city-centric design** is the strongest option for an environmental statistics API.

In a city-centric design, the main resource is the **city**, with related resources such as stations, observations, and analytics nested beneath it. This works well in REST because the relationships are intuitive, for example: cities contain stations, and stations produce observations. It also gives clear, readable endpoint structures and makes the API easier to explain.

Other architectures are possible:

A **station-centric design** makes monitoring stations the primary resource. This is strong when raw sensor collection is the main concern, but it is less intuitive for users who want city-level summaries, trends, or comparisons. It can also make higher-level analytics feel less natural in the API structure.

An **observation-centric design** treats individual measurements as the core resource. This is flexible and maps closely to the database, but it often leads to a lower-level API that feels closer to a data store than a meaningful application. It also makes city-level analysis and presentation harder to communicate clearly.

For the database, a city-centric design still supports a normal relational model: **cities → stations → observations.** This keeps the schema simple while also supporting analytical queries such as averages, trends, anomalies, and city comparisons. It is also the most suitable for a short oral demonstration, because the resource hierarchy is easy to present and the analytical value is immediately visible.

Recommendation: use a **city-centric design.**

Pros:

- clear and intuitive REST structure
- easy to explain relationships
- strong support for city-level analytics
- good fit for demonstration and documentation

Cons:

- some station-level operations may feel secondary
- raw observation ingestion is slightly less central than in lower-level designs

Overall, the city-centric approach gives the best balance between REST clarity, database simplicity, analytical usefulness, and presentation quality.

Associating weather stations with cities

Please tell me the top appropriate weather stations for each of the following cities: London, Manchester, Birmingham, Edinburgh, Glasgow, Leeds, Liverpool, Bristol, Cardiff, Belfast. You can see all available stations here https://www.metoffice.gov.uk/research/climate/maps-and-data/uk-synoptic-and-climate-stations. If there is a city which doesn't have a weather station appropriately close enough, please let me know.

✅ Best mapping (based on YOUR dataset)

This is the **final recommended mapping** — grounded in actual data:

API City	Dataset Station	Notes
London	heathrow	✔ perfect
Manchester	ringway	✔ this IS Manchester Airport
Birmingham	shawbury	⚠ nearest viable
Leeds	bradford	✔ good proxy
Sheffield	sheffield	✔ exact match
Cambridge	cambridge	✔ exact match
Oxford	oxford	✔ exact match
Cardiff	cardiff	✔ exact match
Bristol	yeovilton	⚠ nearest viable
Liverpool	ringway	⚠ reuse Manchester

Scotland

API City	Dataset Station	Notes
Glasgow	paisley	✔ good proxy
Edinburgh	leuchars	⚠ best available

Northern Ireland

API City	Dataset Station	Notes
Belfast	armagh	✔ best available

Import Pathing

How can I import objects into a python file when the file containing the object is in the parent directory?

Add the parent directory to `sys.path`, then import normally:

Python

Run

```python
import sys
import os

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

from parent_file import MyObject
```

Authentication

What would be the best way to implement authorisation into this API, such that only authorised users (admins) can post, put, and delete city, station, and observation data, whereas any user can get and filter it? Should all users create accounts, with admins having special admin accounts? Or should only admins have to create accounts? What technologies would be required? Please explain benefits and limitations and explore alternatives. Please keep your response short and to-the-point.

The best approach is **account-based authentication with role-based authorisation.**

A practical design is:

- all users may access public `GET` endpoints without logging in
- only authenticated users with an **admin role** may use `POST`, `PUT`, and `DELETE`
- regular users, if implemented, would have accounts but no write permissions

For this project, the simplest and strongest version is:

- support **admin accounts only**
- keep read-only access public
- protect all mutation routes with admin authentication

This is the best trade-off because it matches the project's needs without adding unnecessary complexity.

Recommended technologies

- **password hashing** with `bcrypt`
- **JWT bearer tokens** for login sessions
- role field in the user model, e.g. `role = "admin"`
- FastAPI dependency-based route protection

Why this is the best choice

- clear separation between authentication and authorisation
- secure enough for project
- easy to explain and demonstrate
- avoids overengineering a full user-permission system
- aligns well with an API where most users only consume data

Alternative 1: all users create accounts, admins have special accounts

This is more realistic for a larger system, but adds complexity:

- registration flow for normal users
- more documentation and testing
- little benefit if non-admin users only read public data

Alternative 2: only admins have accounts

This is the most suitable option here:

- simplest implementation
- least friction
- enough to demonstrate protected write operations
- public read endpoints remain easy to use

Alternative 3: no accounts, static API key for admins

This is simpler technically, but weaker:

- less secure
- less realistic
- harder to justify as good practice compared with hashed passwords and token-based login

Benefits of the recommended design

- secure password storage
- controlled write access
- simple user model
- easy Swagger testing using bearer tokens

Limitations

- no fine-grained permissions
- no account recovery/email workflows
- less realistic than a full multi-role production system

Overall, the strongest project choice is **JWT-based authentication with admin-only accounts and protected mutation routes, while keeping read-only endpoints public**.